SIMATS ENGINEERING

ASSESSMENT TOOL 2 – DATA INTERPRETATION (MEDIUM)

Course Name: Internet Programming **Course Code**: CSA43

Course Outcome: CO-1 (AT2) **Total Marks**: 90

Name / Reg No: S.Naveen Kumar-192425176

Rubric criterion	Weightage	Where it is addressed
Understanding of Data	23	Section A – interpretation of the given data, its patterns and relationships
Analysis	23	Section B – reasoned, justified answers to the four sub-questions
Application of Concepts	18	Section C – standards, concepts and tools applied to the data
Conclusion	18	Section D – conclusion with insights and implications
Clarity	8	Consistent structure, tables, diagrams and correct terminology throughout

Question 1: HTTP Response Analysis

Given data:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

A response header is a set of metadata fields that tells the client how to read the message that follows. Reading the data field by field shows a clear pattern: three fields are consistent (status line, Content-Type, Connection) and two are defective (**Date** and **Content-Length**).

The defects are related, not isolated. The invalid Content-Length breaks message framing, which in turn affects the persistent connection and makes the success status unreliable:

Invalid Content-Length	→	Body size unknown	→	Keep-alive reuse unsafe	→	200 OK misleading

1. Identify any incorrect or invalid fields in the header.

Field	Value	Valid?	Remark
Status line	HTTP/1.1 200 OK	Yes	Correct format; the status code claims success.
Date	Mon, 01 Apr 2026 10:00:00 GMT	No	1 April 2026 is a Wednesday, not a Monday. The day name and date disagree.
Content-Type	text/html	Yes	Valid media type (a charset parameter is recommended).
Content-Length	-150	No	Must be a non-negative integer; a negative size is meaningless.
Connection	keep-alive	Yes	Valid in HTTP/1.1 (persistent connections are the default).

Invalid fields: Content-Length (negative value) and Date (wrong weekday).

2. Analyze the issue with Content-Length and explain its impact.

Content-Length states the exact size of the message body in bytes. It is defined as a sequence of decimal digits, so it can never be negative. Because -150 is invalid, the receiver cannot work out where the body ends. The impact follows step by step:

- The client must treat the response as malformed and may reject it, show an error, or discard the body.
- Connection is keep-alive, so the connection is meant to be reused. With broken framing the client cannot tell where this response ends and the next begins, so the connection has to be closed and the benefit of persistence is lost.

- The page may be truncated, or the browser may wait for bytes that never arrive.
- Different length handling between a proxy and a server can cause request/response desynchronisation, a recognised security risk (HTTP request smuggling).

3. Determine whether the status code matches the response correctness.

It does not match. The status 200 OK tells the client that the request succeeded and that the response is valid and complete. The header, however, is malformed, so a 200 status is misleading. If the server cannot produce a valid response, it should return **500 Internal Server Error** to signal the server-side fault. The code 200 is appropriate only when the header and body are both correct.

4. Suggest corrections for the identified errors.

- Set Content-Length to the true, non-negative byte size of the body (for example 150), or remove it and use Transfer-Encoding: chunked.
- Correct the Date header so the weekday matches the date: Wed, 01 Apr 2026.
- Add a character set to Content-Type: text/html; charset=UTF-8.

Corrected response header:

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
Connection: keep-alive
```

- **Message syntax and framing (RFC 9110, RFC 9112):** Content-Length is one or more decimal digits giving the body size in octets, which is why -150 fails. Framing (Content-Length or chunked encoding) is what makes keep-alive reuse possible.
- **Date format:** the Date header uses the fixed IMF-fixdate format, in which the day name should agree with the calendar date.
- **Status code classes:** 2xx means success and 5xx means server error, so the correct class for a failed response is 5xx
- **Tools:** the command curl -i URL shows the raw status line and headers; wc -c index.html gives the exact byte count to use as Content-Length; the browser DevTools Network tab shows response headers and failures.

The header has three correct fields and two defective ones. The Content-Length error is the most serious because message framing underpins keep-alive reuse and the meaning of 200 OK.

Implications: (1) servers and frameworks should calculate Content-Length automatically instead of setting it by hand; (2) a response that cannot be built correctly must carry a 5xx status, not 200; (3) proxies and servers should parse headers strictly, because inconsistent framing is the basis of request-smuggling attacks.

Question 2: DOM Structure Analysis

Given data:

```
<html>
    <head>
        <title>TestPage</title>
    </head>
    <body>
        <div>
            <p>HelloWorld
        </div>
    </body>
</html>
```

The listing is a tree: html contains head and body, head contains title, and body contains div, which contains p. Checking every element for a matching start and end tag gives the following pattern:

Element	Start tag	End tag	Status
<html>	Yes	Yes	Correct
<head>	Yes	Yes	Correct
<title>	Yes	Yes	Correct
<body>	Yes	Yes	Correct
	Yes	Yes	Correct
	Yes	**No**	**Unclosed** (missing)

Five of the six elements are balanced and correctly nested. The hierarchy is right, so the error is a missing end tag, not a nesting error. Document-level items are also missing: the DOCTYPE, the lang attribute and the charset declaration.

1. Identify missing or improperly closed elements.

- The ** ** element has no closing tag. HTML5 lets a browser close a paragraph implicitly when its parent ends, but in the source it is still unclosed, and it is a fatal error in XHTML/XML.
- The **<!DOCTYPE html>** declaration is missing.
- The <html> element has no lang attribute and the <head> has no <meta charset> (best-practice omissions).
- "HelloWorld" has no space; it is probably meant to be "Hello World".

2. Analyze how the browser will interpret this structure.

HTML parsers are error tolerant. When the browser reaches while the is still open, it inserts the missing (an implied end tag) and then closes the div. The DOM that results is:

```
html
    head
        title → "TestPage"
    body
        div
            p → "HelloWorld"
```

Because the DOCTYPE is missing, the browser also switches to **quirks mode**, which imitates old layout rules. If the same markup were parsed as XHTML/XML, the unclosed would be a well-formedness error and the page would not render at all.

3. Determine the impact on rendering and layout.

- For this small page the visible output is almost unchanged: "HelloWorld" appears in a paragraph with default margins.
- Quirks mode can alter the box model, default font sizes and element heights, so layout may differ between browsers.
- If more content follows the unclosed , automatic closing may place elements in unexpected parents and break the layout or CSS selectors.

- The DOM no longer matches the author's intention, which makes JavaScript DOM manipulation and debugging harder.

4. Suggest corrections to make the DOM valid.

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <title>Test Page</title>
    </head>
    <body>
        <div>
            <p>Hello World</p>
        </div>
    </body>
</html>
```

- **HTML parsing algorithm (WHATWG):** the end tag of p is optional and is inserted automatically when the parent closes (implied end tag).
- **Standards vs quirks mode:** the DOCTYPE selects standards mode. In JavaScript, document.compatMode returns "BackCompat" in quirks mode and "CSS1Compat" in standards mode.
- **XML well-formedness:** XHTML and XML require every start tag to have a matching end tag; HTML does not enforce this.
- **Tools:** the W3C Nu HTML Checker reports the missing DOCTYPE, and the DevTools Elements panel shows the repaired DOM.

The markup has one structural defect (the unclosed <p>) and one document-level defect (no DOCTYPE). The browser repairs the first silently, so the page looks correct and the problem stays hidden.

Implications: a page that renders correctly is not necessarily valid. Relying on error recovery makes the layout depend on the browser's guessing rules, so developers should always declare the DOCTYPE, close elements explicitly and validate markup, especially because the page may later be extended or read by XML tools.

Question 3: GET vs POST in Login System

Given data:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Both methods send the same login data; the only difference is where the data travels. With GET the credentials are part of the URL, a place that browsers, servers and proxies display, store and log. With POST they move into the request body, which none of these places show by default. The security ratings follow this pattern: the farther the credentials are from places where they are displayed or stored, the higher the rating.

GET: data in URL (**Low**) → POST: data in body (**Moderate**) → POST + HTTPS + hashing (**High**)

POST is rated only "Moderate" because the body is hidden but still travels as readable text unless HTTPS encrypts it. The third level shows what full protection needs.

1. Interpret the differences in data transmission between GET and POST.

Parameter	GET	POST
Data location	Appended to the URL as a query string	Sent inside the HTTP request body
Visibility	Visible in the address bar	Not shown in the URL
History / bookmarks	Stored in history; can be bookmarked	Not stored in history
Server / proxy logs	Full URL, including data, is logged	Body is normally not logged
Length limit	Limited by URL length	No practical limit
Caching	May be cached	Not cached by default
Intended use	Retrieving data (safe, idempotent)	Submitting data that changes state

GET exposes the data in the URL, whereas POST carries it in the body. POST only hides the data from the URL; it does not encrypt it.

2. Analyze which method is more suitable for login and why.

POST is more suitable. A login request carries secrets and creates a session, so it is a state-changing operation. POST keeps the username and password out of the URL, browser history, bookmarks, server logs and the Referer header, and it avoids URL length limits and accidental caching. GET is meant for harmless, repeatable retrieval and should never carry secrets.

3. Identify security risks in using GET for authentication.

- **Credential exposure:** the password is visible on screen (shoulder surfing) and in the address bar.
- **History and bookmarks:** the URL with the credentials stays in browser history and can be bookmarked or synced.
- **Log leakage:** web server, proxy and load balancer logs record the full URL, so passwords are stored in plain text.
- **Referer leakage:** the URL may be sent to third-party sites in the Referer header.
- **Caching and sharing:** the URL can be cached, copied and replayed.
- **CSRF risk:** a login can be triggered by loading a crafted link or image.

4. Suggest best practices for secure login implementation.

- Use **POST** over **HTTPS (TLS)** only, and enable HSTS.
- Never store plain-text passwords; use a salted, slow hash such as bcrypt, scrypt or Argon2.
- Use parameterised queries against SQL injection and validate input on the server.
- Add CSRF tokens, rate limiting, account lockout or CAPTCHA against brute force.
- Offer multi-factor authentication where possible.
- Set the Secure, HttpOnly and SameSite flags on session cookies and regenerate the session ID after login.
- Show generic error messages ("Invalid username or password") and never log credentials.

HTTP method semantics (RFC 9110): GET is safe and idempotent and is meant for retrieval; POST is meant for submitting data for processing.

- **Transport security:** TLS (HTTPS) encrypts the request in transit, but only POST also keeps the data out of history and logs, so both are needed together.

- **OWASP guidance:** the Authentication and Password Storage Cheat Sheets recommend POST, HTTPS, salted slow hashing, rate limiting and MFA.

A secure login form applying these ideas:

```
<form method="post" action="/login">
    <input type="text" name="user" autocomplete="username">
    <input type="password" name="pass" autocomplete="current-password">
    <button type="submit">Login</button>
</form>
```

GET is unsuitable for authentication because it leaks credentials into the address bar, history, logs and Referer header. POST is the correct method, but it is only the first layer of protection.

Implications: real security comes from combining POST with HTTPS, hashed passwords and attack limiting. Using GET for a login is therefore both a design error and a compliance risk, because passwords stored in logs can be exposed to anyone who reads them.

Question 4: HTML Code Inspection

Given data:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Checking each element for start and end tags shows the pattern in the table below. The structural elements are fine; the problems lie in the content inside the body.

Element	Start tag	End tag	Status
<html>, <head>, <title>, <body>	Yes	Yes	Correct
<h1>	Yes	**No**	Missing </h1>
	Yes	**No**	Missing
<img>	Yes	Not required (void)	**alt** attribute missing

The two unclosed elements are adjacent block-level elements. Because <h1> stays open, the paragraph and image that follow are pulled into it, so the missing end tags change the **structure** of the page, not only its appearance. The tree the browser builds is:

```
body
    h1
        "Welcome"
        p
            "This is a paragraph"
            img (src="image.jpg")
```

1. Identify missing closing tags.

- **</h1>** is missing after "Welcome".
- ** ** is missing after "This is a paragraph".

The <img> tag is a void element and needs no closing tag in HTML5.

2. Analyze issues with the <img> tag.

- The **alt** attribute is missing. It is required for valid HTML and essential for accessibility (screen readers) and for showing text when the image fails to load.
- No width and height are given, so the browser cannot reserve space and the layout may shift as the image loads.
- In XHTML the tag must be self-closed (); in HTML5 no end tag is needed.
- The image sits inside the unclosed paragraph and therefore inside the heading, which is a structural error.

3. Determine how browsers will handle these errors.

Browsers do not reject the page; they apply the error-recovery rules of the HTML5 parsing algorithm. A <p> start tag does not close an open <h1>, so the paragraph becomes a child of the heading and the image a child of the paragraph (see the tree in Section A). All open elements are closed only when </body> is reached. As a result the paragraph text is displayed in the large, bold heading style and the structure differs from the author's intention. The image still loads, but without alt text it has no fallback. No error is shown to the user, so such mistakes often go unnoticed.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sample</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph</p>
    <img src="image.jpg" alt="Sample image" width="300" height="200">
</body>
</html>
```

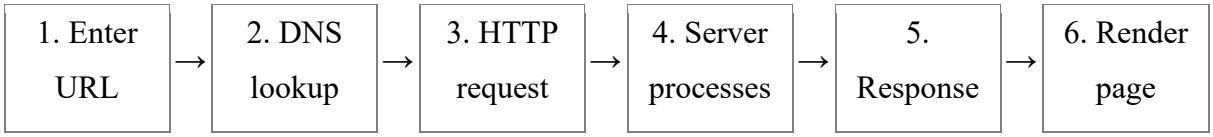
- **HTML parsing algorithm (WHATWG):** error recovery builds a tree even from broken markup, and a p start tag does not implicitly close an open h1.
- **Content models:** a heading may contain only phrasing content, and p is flow content, so a paragraph inside h1 is invalid.
- **Void elements:** img, br, input and similar elements never take an end tag in HTML5.
- **Accessibility (WCAG 2.1, success criterion 1.1.1):** non-text content such as images needs a text alternative, supplied by the alt attribute.
- **Tools:** the W3C Nu HTML Checker reports the structural errors, the DevTools Elements panel reveals the actual nesting, and a Lighthouse audit flags images without alt text.

The snippet has two missing closing tags and an incomplete <img> tag. Browsers hide these mistakes by guessing the structure, but the guess nests the paragraph and image inside the heading.

Implications: unclosed elements can silently change both the layout and the meaning of a page, and a missing alt attribute excludes users of assistive technology. Closing every element, adding alt text and validating the markup gives a page that is valid, accessible and predictable in every browser.

Question 5: Browser-Server Communication Flow

Given data: the six-step sequence of a web request, shown below.



The six steps form a strictly sequential chain: each step needs the output of the one before it (URL, then IP address, then connection, then request, then response, then page). Two stages are implicit in the list: the **TCP/TLS handshake** between steps 2 and 3, and the **extra requests** for images, CSS and JavaScript after step 5. The total load time is therefore the sum of the delays in every stage.

1. Interpret each step in the communication flow.

Step	Stage	Explanation
1	User enters a URL	The browser parses the URL into protocol (http/https), host name, port and path, and checks its own cache for a stored copy.
2	DNS lookup	The host name is converted into an IP address. The browser cache, operating-system cache and resolver are checked in turn; if not found, the resolver queries the root, TLD and authoritative name servers.
3	HTTP request is sent	After a TCP connection (and a TLS handshake for HTTPS) is set up with the server's IP, the browser sends a request such as GET /index.html with headers (Host, User-Agent, Accept, Cookies).

Step	Stage	Explanation
4	Server processes request	The web server receives the request, routes it, runs application logic, queries databases or reads files, and prepares the response.
5	Response is returned	The server sends the status line (for example 200 OK), headers (Content-Type, Content-Length, caching headers) and the body (HTML).
6	Browser renders page	The browser parses HTML into the DOM and CSS into the CSSOM, combines them into a render tree, performs layout and paints the page. It also fetches the images, CSS and JavaScript the page refers to.

2. Identify where delays can occur.

- **DNS lookup:** an uncached lookup needs several round trips to different name servers.
- **Connection setup:** the TCP three-way handshake and TLS negotiation each add round-trip time.
- **Network latency and bandwidth:** long distance to the server, congestion or large responses slow the transfer.
- **Server processing:** slow database queries, heavy computation or an overloaded server increase the time to first byte.
- **Rendering:** render-blocking CSS and JavaScript, large pages and many extra requests delay the first paint.

3. Analyze the role of DNS in the process.

DNS (Domain Name System) works like the Internet's phone book. Computers communicate using IP addresses, while people use readable domain names. Without DNS the browser would not know which server to contact, so no request could be sent; it is the first network step after the URL is entered. A resolver looks the name up through a hierarchy (root server, then TLD server such as .com, then the authoritative server) and returns the IP address, which is cached for the time set by the record's TTL. Every new host name needs a lookup, so DNS speed directly affects page-load time, and a DNS failure makes a site unreachable even when its server is working.

4. Suggest optimizations to improve performance.

Each delay point has a matching optimization:

Delay point	Optimization
DNS lookup	Fast DNS provider, suitable TTL values, dns-prefetch and preconnect hints
Connection setup	HTTP/2 or HTTP/3, keep-alive connections, TLS session reuse
Distance and transfer size	CDN close to users; gzip or Brotli compression; compressed and resized images
Server processing	Optimised database queries, server-side caching, load balancing
Repeat visits	Cache-Control and ETag headers so unchanged files are not downloaded again
Rendering	Minified files, async or defer for scripts, inline critical CSS, lazy-loaded images, fewer requests

- **DNS records and caching:** A and AAAA records map names to IPv4 and IPv6 addresses, CNAME records create aliases, and the TTL controls how long an answer is cached.
- **HTTP versions:** HTTP/1.1 keep-alive reuses a connection, while HTTP/2 multiplexes many requests over one connection and HTTP/3 reduces handshake delay further.
- **Browser rendering pipeline:** HTML to DOM, CSS to CSSOM, render tree, layout, paint.
- **Tools:** nslookup or dig shows DNS answers (dig also prints the query time); the DevTools Network Timing tab splits a request into DNS lookup, initial connection, SSL, waiting (time to first byte) and content download; Lighthouse or PageSpeed Insights measures the page and recommends fixes